🧩 **Fragenkatalog – JavaScript-Tutorial**
Fachgespräche – 2. Klasse Medientechnik | HTBLA Traun
Prof. DI Martin Zoister

🔹 **Kapitel 01: Einführung & Ausgabe**

1. **Wozu verwendet man `console.log()`?**
   → Zur Ausgabe von Informationen **in der Entwickler-Konsole** – hauptsächlich zum Debuggen.

2. **Warum sollte man `document.write()` vermeiden?**
   → Es **löscht die ganze Seite**, wenn es nach dem Laden aufgerufen wird – das ist gefährlich.

3. **Wie gibt man den Inhalt einer Variable auf der Webseite aus?**
   → Mit `document.getElementById("id").innerHTML = wert;`.

4. **Was macht `prompt()` und wie ist die Benutzererfahrung?**
   → Es öffnet ein Eingabefenster – **blockiert aber die ganze Seite**, deshalb nur für Lernzwecke.

5. **Warum ist `innerHTML` die professionelle Methode?**
   → Weil man nur **einen Teil der Seite** ändert, ohne alles neu zu laden.

🔹 **Kapitel 02: Variablen & Datentypen**

1. **Wann verwendet man `const` und wann `let`?**
   → `const` für Werte, die **nicht geändert** werden; `let` nur, wenn der Wert sich ändern muss.

2. **Was ist der Unterschied zwischen `"5"` und `5`?**
   → `"5"` ist ein **String** (Text), `5` ist eine **Zahl** – und beide verhalten sich bei Berechnungen anders.

3. **Warum beginnt der Array-Index bei `0`?**
   → Weil JavaScript (wie viele Sprachen) **0-basiert** zählt – das erste Element ist also bei `[0]`.

4. **Was liefert `typeof null` und warum ist das seltsam?**
   → `"object"` – das ist ein **historischer Fehler**, den man nicht ändern kann, weil sonst altes Code kaputtgeht.

5. **Wozu braucht man Template-Literale (Backticks)?**
   → Um **Variablen direkt in Texte einzufügen** – z. B. `` `Hallo ${name}` `` – das ist lesbarer als `"Hallo " + name`.

🔹 **Kapitel 03: Operatoren**

1. **Warum ist `===` besser als `==`?**
   → `===` vergleicht **Wert und Typ**, `==` macht heimliche Typumwandlung – das führt zu Fehlern.

2. **Was macht der ternäre Operator?**
   → Er ist eine **Kurzschreibweise** für `if...else`: `bedingung ? wahr : falsch`.

3. **Wann verwendet man `+=`?**
   → Um einen Wert **zum bestehenden Wert hinzuzufügen** – z. B. `punkte += 10` statt `punkte = punkte + 10`.

4. **Was ist der Unterschied zwischen `&&` und `||`?**
   → `&&` = **beides muss wahr sein**; `||` = **mindestens eines muss wahr sein**.

5. **Warum ergibt `"5" + 3` den Wert `"53"`?**
   → Weil JavaScript bei `+` **vermutet, dass es sich um Text handelt**, und den Zahl-Teil in einen String umwandelt.

🔹 **Kapitel 04: Bedingungen**

1. **Was passiert, wenn die erste `if`-Bedingung `true` ist?**
   → Alle folgenden `else if` und `else` werden **übersprungen**.

2. **Warum braucht man geschweifte Klammern `{}`?**
   → Damit **mehrere Zeilen** zur Bedingung gehören – ohne Klammern gehört nur die erste Zeile dazu.

3. **Wann verwendet man `else if` und wann mehrere `if`?**
   → Bei **sich ausschließenden Fällen** → `else if`; bei **unabhängigen Prüfungen** → mehrere `if`.

4. **Kann man `&&` und `||` in einer Bedingung mischen?**
   → Ja, aber man sollte **Klammern setzen**, damit klar ist, was zuerst geprüft wird.

5. **Was bedeutet „Fallthrough“ bei `switch`?**
   → Ohne `break` **fällt der Code in den nächsten Fall** – das ist meistens ein Fehler.

🔹 **Kapitel 05: Schleifen**

1. **Wann verwendet man eine `for`-Schleife?**
   → Wenn man **genau weiß, wie oft** etwas wiederholt werden soll (z. B. 5 Durchläufe).

2. **Was passiert bei einer Endlosschleife?**
   → Der Browser **hängt sich auf**, weil er endlos rechnet – meistens wegen fehlender Abbruchbedingung.

3. **Wann ist `do...while` sinnvoll?**
   → Wenn der Code **mindestens einmal** ausgeführt werden muss – auch wenn die Bedingung am Anfang falsch ist.

4. **Warum braucht man bei `setInterval` eine ID?**
   → Damit man den Timer **später stoppen** kann – sonst läuft er ewig im Hintergrund.

5. **Was ist der Unterschied zwischen `while` und `for`?**
   → `while` prüft **nur eine Bedingung**, `for` hat **Initialisierung, Bedingung und Update** in einer Zeile.

🔹 **Kapitel 06: Arrays**

1. **Wie greift man auf das letzte Element eines Arrays zu?**
   → Mit `arr[arr.length - 1]`, weil der Index bei `0` beginnt.

2. **Was macht `push()`?**
   → Fügt ein Element **am Ende** des Arrays hinzu.

3. **Warum ist `const arr = [...]` erlaubt, obwohl man das Array verändern kann?**
   → Weil `const` nur die **Variable** schützt – nicht den **Inhalt** des Arrays.

4. **Was passiert, wenn man `arr[100] = "Test"` schreibt?**
   → Es entstehen **„Löcher“** im Array – alle dazwischen liegenden Elemente sind `undefined`.

5. **Warum verwendet man Schleifen mit Arrays?**
   → Weil man so **alle Elemente nacheinander verarbeiten** kann – z. B. für Durchschnittsberechnung.

🔹 **Kapitel 07: Funktionen**

1. **Warum ist eine Funktion mit Rückgabewert besser als eine mit Seiteneffekt?**
   → Weil sie **wiederverwendbar, testbar und vorhersagbar** ist – sie verändert nichts außerhalb.

2. **Was ist `this` in einer Objekt-Methode?**
   → Es verweist auf **das Objekt, zu dem die Methode gehört**.

3. **Warum verwendet man Parameter bei Funktionen?**
   → Damit die Funktion **mit verschiedenen Werten arbeiten** kann – sie wird dadurch flexibel.

4. **Was ist eine „Closure“?**
   → Eine innere Funktion, die **auf Variablen der äußeren Funktion zugreifen** kann – auch nach deren Beendigung.

5. **Warum sollte man Funktionen kurz und fokussiert halten?**

→ Weil sie dann **besser lesbar, wartbar und fehlerfrei** sind.

🔹 **Kapitel 08: Objekte**

1. **Wann verwendet man ein Objekt statt eines Arrays?**
   → Wenn man **unterschiedliche Eigenschaften** (Name, Alter, Noten) zu einem Thema zusammenfassen will.

2. **Warum braucht man `this` in Methoden?**
   → Um auf **andere Eigenschaften desselben Objekts** zuzugreifen.

3. **Kann man einem Objekt nachträglich Eigenschaften hinzufügen?**
   → Ja – JavaScript-Objekte sind **dynamisch**.

4. **Was ist der Vorteil einer Methode im Objekt?**
   → Sie gehört **zum Objekt** und kann direkt auf dessen Daten zugreifen – das ist strukturiert.

5. **Warum ist `schueler.name` lesbarer als `schueler[0]`?**
   → Weil der **Name aussagt, was gemeint ist** – bei Index 0 muss man raten.

🔹 **Kapitel 09: DOM-Manipulation**

1. **Warum ist `textContent` sicherer als `innerHTML`?**
   → Weil es **HTML-Code nicht interpretiert** – so kann kein bösartiger Script-Code ausgeführt werden.

2. **Was macht `document.querySelector()`?**
   → Es findet **das erste Element**, das zu einem CSS-Selektor passt (z. B. `#id` oder `.klasse`).

3. **Warum sollte man neue Elemente im Speicher bauen, bevor man sie einfügt?**
   → Weil das **schneller und performanter** ist – der Browser muss nicht bei jedem Schritt die Seite neu zeichnen.

4. **Was ist der Unterschied zwischen `getElementById` und `querySelector`?**
   → `getElementById` sucht **nur nach ID**, `querySelector` kann **alle CSS-Selektoren** verwenden.

5. **Warum braucht man bei dynamischen Elementen Event-Delegation?**
   → Weil man **nicht für jedes neue Element einen Listener hinzufügen** möchte – stattdessen hört ein übergeordnetes Element zu.

🔹 **Kapitel 10: Ereignisse & Timing**

1. **Warum ist `addEventListener` besser als `onclick`?**
   → Weil man **mehrere Funktionen** auf ein Ereignis legen kann – `onclick` überschreibt alte Funktionen.

2. **Was passiert, wenn man `setInterval` nicht mit `clearInterval` stoppt?**
   → Die Funktion läuft **ewig im Hintergrund** – das verbraucht Speicher und Rechenleistung (Speicherleck).

3. **Warum braucht man eine Intervall-ID?**
   → Um später mit `clearInterval(id)` den Timer **gezielt stoppen** zu können.

4. **Was ist im `event`-Objekt enthalten?**
   → Informationen zum Ereignis – z. B. welche Taste gedrückt wurde oder welches Element geklickt wurde.

5. **Warum ist Event-Delegation bei dynamischen Inhalten wichtig?**
   → Weil direkt angehängte Listener **nicht funktionieren**, wenn das Element erst später hinzugefügt wird.

> 💡 **Tipp für das Fachgespräch**:
> Frage **erst**, ob der/die Schüler:in die **eigene Lösung erklären** kann.
> Erst dann stelle eine dieser Fragen – als **Unterstützung oder Vertiefung**.

© HTBLA und Fachschule für IT, Traun – Medientechnik
Erstellt am: Dezember 2025